COMPUTACION 2021

Clase 3

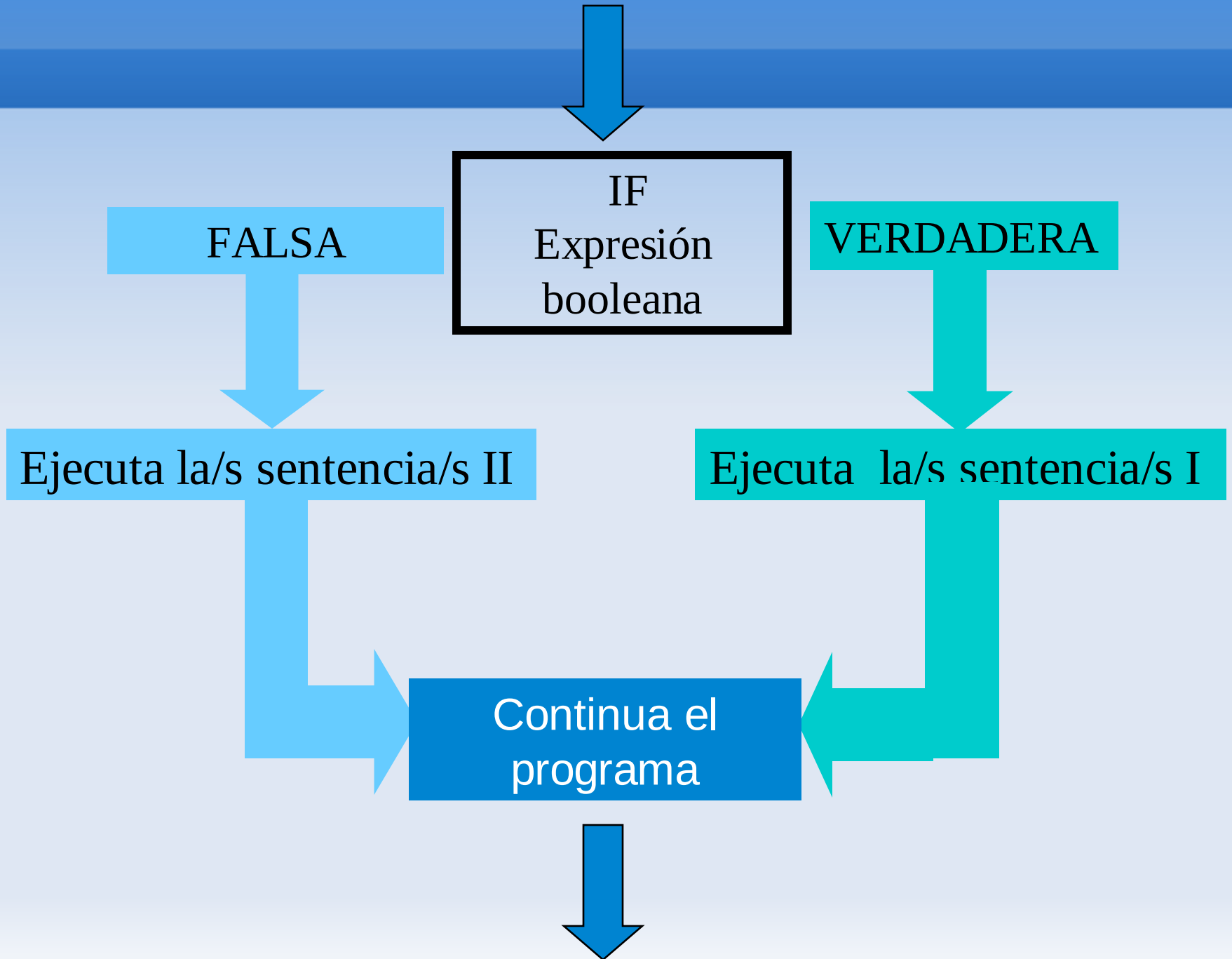
Temas de la clase 3

- Sentencia IF -THEN - ELSE
- Sentencias IF anidadas
- Sentencia CASE : bifurcación múltiple

Recordemos el ejemplo de la clase 1

Dados 2 números enteros (cualesquiera), se desea detectar los que son positivos y escribirlos.

ALGORITMO DECISION
  Ingresar **a,b**
  Si **a>0** entonces escribir **a**
  Si **b>0** entonces escribir **b**
Fin del algoritmo DECISION


IF
Expresión
booleana
FALSA
VERDADERA
Ejecuta la/s sentencia/s II
Ejecuta la/s sentencia/s I
Continua el
programa

Sentencia IF-THEN-ELSE

- La sentencia IF especifica que **sentencia I** (que sigue a THEN) debe ejecutarse sólo si una cierta condición (expresión lógica) es verdadera.
- Si es falsa, entonces se ejecuta **sentencia II**, que sigue al símbolo ELSE (si existe)

De aquí en mas se usará la palabra **sentencia** para indicar tanto una sentencia simple como compuesta (o estructurada)

Sentencia de decisión

- Una sentencia de decisión puede contener otras sentencias (sentencia compuesta)
- Su función principal es decidir qué sentencia/s se debe ejecutar de acuerdo al resultado de una expresión.
- Luego de evaluar la expresión lógica, se ejecuta la sentencia que sigue a THEN o la sentencia que sigue a ELSE , pero NO AMBAS.

Sintaxis de la sentencia IF-THEN-ELSE

```
IF  expresión booleana THEN
        sentencia1
 ELSE
        sentencia 2;
```

ó

Ejemplo

Leer un par de coordenadas (x,y) de un punto del plano. Verificar si el punto se encuentra en el círculo centrado en el origen y con radio 5.

```
Program circun;
 Var
  x,y,d:real;
Begin
 write('Ingrese dos valores reales');
 readln(x,y);
 d:=x*x+y*y;
  IF d<=25 THEN
  begin
      writeln('punto dentro circ.');
      writeln('x*x+y*y=',d)
  end
  ELSE
      writeln('pto fuera de la circ.')
End.
```

Sentencia compuesta: debe encerrarse entre los delimitadores begin y end Significa que si la condición es verdadera deben ejecutarse dos sentencias

Se debe colocar ; para separar las dos sentencias

No se debe escribir punto y coma (;). Aquí no termina la sentencia IF-THEN-ELSE.

- Si sólo interesa ejecutar una sentencia cuando la expresión booleana es verdadera, se usa una estructura IF simplificada.
- Esta sentencia simplificada sólo posee la sentencia que sigue a THEN:

```
IF  expresión booleana THEN
      sentencia;
```

Ejemplo de IF-THEN

Dada una ecuación cuadrática, calcular y mostrar las raíces complejas (se muestra sólo parte de un programa).

```
d:= b*b - 4*a*c ;
IF d < 0 THEN
begin
  R:= -b/(2*a);
  I:= sqrt(abs(d))/(2*a);
  writeln('x=', R:6:2,'+/-',I:6:2,' i')
end;
```

SENTENCIAS CONDICIONALES ANIDADAS

Las Sentencias dentro de IF o ELSE pueden ser una sentencia simple o una estructurada incluso puede ser otra estructura de decisión. En este caso tendremos sentencias IF anidadas.

Sintaxis

```
IF        expresión lógica        THEN
      begin
          sentencia/s (IF,asignación,E/S)
      end
  ELSE
      begin
          sentencia/s (IF,asignación,E/S)
      end;
```

Ejemplo

Analizar si la suma de dos números cualesquiera es positiva o negativa. En caso de ser cero, indicar que estos son números opuestos.

```
Program anidamiento;
    Var
        a,b,suma:real;
    Begin
        Readln(a,b);
        suma:=a+b;
        IF suma>0 THEN
          writeln('suma Positiva')
        ELSE
            IF suma<0 THEN
                writeln('suma  Negativa')
            ELSE
              writeln ('numeros Opuestos');
    End.
```

Observación!

- En el ejemplo anterior hemos tenido 3 condiciones mutuamente excluyentes:
    - suma>0
    - suma<0
    - suma=0
- Cada una de ellas desata acciones distintas. En estos casos el anidamiento de IF bien desarrollado evita preguntas superfluas (por ejemplo preguntar suma=0, ya que es la única opción que hay en el segundo ELSE en ese caso)

Comparación entre decisión secuencial y decisión anidada

Supongamos que el problema anterior lo desarrollamos con sentencias de decisión secuenciales

```
IF suma>0 THEN
    writeln('suma Positiva');
IF suma<0 THEN
    writeln('suma Negativa');
IF suma=0 THEN
      writeln('numeros Opuestos');
```

Es ineficiente !!!

EL BUEN USO DE LAS EXPRESIONES LOGICAS

Podemos aumentar la legibilidad de un programa usando apropiadamente las expresiones booleanas.

Ejemplo:

Imprimir el mínimo de tres números a,b,c.

Podemos usar dos IF anidados

```
IF a<b THEN
    IF a<c THEN
        writeln(a);
```

O bien definir una expresión booleana más compleja y usar UN solo IF:

```
IF (a<b) and (a<c) THEN
    writeln(a);
```

Observación!

A fin de evitar la ambigüedad sintáctica que surge de la construcción:

```
IF exp_booleana1 THEN IF exp_booleana2 THEN
            sentencia1 ELSE sentencia2;
```

Conviene usar la ***indentación*** a fin de incrementar la legibilidad y evitar la ambigüedad. Nos queda la construcción equivalente**:**

```
IF exp_booleana1 THEN
      IF exp_booleana2 THEN
            sentencia1
       ELSE
            sentencia2;
```

SENTENCIA CASE : bifurcación múltiple

Cuando tenemos varias sentencias mutuamente excluyentes podemos usar CASE (con algunas restricciones).

Ejemplo

En una empresa de servicios de internet y cable se cuenta con diferentes paquetes para ofrecer a los clientes, que se muestran a continuación. Se desea indicarle a una persona lo que debe pagar según el plan elegido.

Categoría	Tipo de Plan	Precio
S	Solo Internet 5M	$1700
C	Solo Cable	$1700
D	Internet 3M y Cable	$2100
X	Internet 5M y Cable	$2500

Programa PASCAL usando IF anidados

```
Program Planes;
    Var
      plan:char;
      pago:real;
      error:boolean;
```

```
Begin
Error:=false;
Writeln('Ingrese plan');
Readln(plan);
   IF (plan = 'S') or  (plan='C') THEN
      pago := 1700
  ELSE
     IF plan = 'D' THEN
        pago :=2100
     ELSE
         IF plan = 'X' THEN
            pago := 2500
                   ELSE
              error:=true;
IF error THEN
      Writeln('Plan inexistente')
ELSE
      Writeln('El valor a pagar es = $',pago:6:2)
End.
```

Selección múltiple: CASE-OF

La sentencia CASE permite seleccionar una de entre varias sentencias según un selector.

El selector es una **expresión ordinal**.

<u>No puede ser del tipo real ni cadena</u>

Sintaxis sentencia CASE

CASE selector **OF**

Lista de ctes.1 : sentencia1;

Lista de ctes.2 : sentencia2;

.....

Lista de ctes.n : sentencian;

ELSE sentencia m

END;

```
Program  planes;
    Var
       plan:char;
       pago:real;
      error:boolean;
Begin
  Error:=false;
  Writeln('Ingrese plan');
  Readln(plan);
  CASE plan OF
         'S','C' :pago := 1700;
           'D' : pago:=2100;
            'X' : pago:=2500;
    ELSE
            error:=true
 END;
 IF error THEN
      Writeln('Plan inexistente')
 ELSE
      Writeln('El valor a pagar es = $',pago:6:2)
End.
```

Los IF anidados pueden ser llevados a la sentencia CASE equivalente:

Observaciones

- CASE selecciona como sentencia/s a ejecutar aquella/s cuya etiqueta (las ctes.) es igual o contenga al valor actual del selector.
- Si el valor actual de selector (que es una expresión) no encuentra un valor igual en alguna de las listas de constantes se ejecuta la sentencia que sigue a ELSE (si existe)
- El tipo de dato de la expresión selector debe ser ordinal (*se excluye el tipo real y string*).
- Cada constante debe ser única dentro de las listas.
- No usar intervalos de la forma 2.5<x<7.4 como etiquetas (en la lista de constantes)